Лабораторная работа №8

Целочисленная арифметика многократной точности

12 Марта 2026 - Сунь Маосин

2026

Российский университет дружбы народов, Москва, Россия

Цель работы

Цель работы

В данной лабораторной работе я реализовал пять алгоритмов целочисленной арифметики многократной точности: сложение, вычитание, умножение столбиком, быстрое умножение и деление. Эти алгоритмы позволяют обрабатывать числа любой длины, представляя их в виде массивов цифр в системе счисления с основанием $b = 10$.

Вспомогательные функции

Вспомогательные функции

Для работы с числами как с массивами цифр были реализованы функции преобразования строк в списки цифр и обратно: `str_to_digits` и `digits_to_str`.

```
def str_to_digits(s):
    """Convert string of digits to list of integers"""
    return [int(ch) for ch in s]

def digits_to_str(digits):
    """Convert list of digits to string"""
    return ''.join([str(d) for d in digits])
```

Рис. 1: Вспомогательные функции

Алгоритм 1: Сложение

Описание алгоритма

Алгоритм выполняет поразрядное суммирование, имитируя расчёт «в столбик». Процесс начинается с младших разрядов, переменная k отвечает за перенос в старший разряд.

Код реализации

```
def add(u, v, b=10):
    """
    Algorithm 1: Addition of non-negative integers
    u, v: strings of digits (same length)
    b: base (default 10)
    Returns: sum as string
    """
    # Convert strings to lists of digits
    u_digits = str_to_digits(u)
    v_digits = str_to_digits(v)
    n = len(u_digits)

    # Initialize
    j = n - 1    # index from rightmost digit (0-based)
    k = 0          # carry
```

Результат выполнения

```
1. ADDITION
12345 + 56789 = 69134
Expected: 69134
Correct: True
```

Рис. 3: Результат сложения $12345 + 56789 = 69134$

На примере $12345 + 56789 = 69134$ алгоритм работает корректно, что подтверждается выводом `Correct: True`.

Алгоритм 2: Вычитание

Описание алгоритма

Алгоритм реализует операцию $u - v$ при условии $u > v$. Переменная k используется для фиксации займа из старшего разряда.

Код реализации

```
def subtract(u, v, b=10):
    """
    Algorithm 2: Subtraction of non-negative integers (u > v)
    u, v: strings of digits (same length, u > v)
    b: base (default 10)
    Returns: difference as string
    """
    # Convert strings to lists of digits
    u_digits = str_to_digits(u)
    v_digits = str_to_digits(v)
    n = len(u_digits)

    # Initialize
    j = n - 1    # index from rightmost digit
    k = 0          # borrow
    w_digits = [0] * n
```

Результат выполнения

```
2. SUBTRACTION
56789 - 12345 = 44444
Expected: 44444
Correct: True
```

Рис. 5: Результат вычитания $56789 - 12345 = 44444$

Пример $56789 - 12345 = 44444$ демонстрирует правильную работу алгоритма с обработкой займов.

Алгоритм 3: Умножение столбиком

Описание алгоритма

Классическая реализация длинного умножения через вложенные циклы. Внешний цикл фиксирует цифру множителя, внутренний перемножает её со всеми цифрами множимого с последующим суммированием со сдвигом.

Код реализации

```
def multiply_column(u, v, b=10):
    """
    Algorithm 3: Multiplication by column
    u, v: strings of digits
    b: base (default 10)
    Returns: product as string
    """
    u_digits = str_to_digits(u)
    v_digits = str_to_digits(v)
    n = len(u_digits)
    m = len(v_digits)

    # Initialize result array
    w_digits = [0] * (m + n)

    # Process each digit of v from right to left
    for j in range(m-1, -1, -1):
        if v_digits[j] == 0:
            continue
```

Результат выполнения

```
3. MULTIPLICATION (COLUMN)
123 × 45 = 5535
Expected: 5535
Correct: True
```

Рис. 7: Результат умножения $123 \times 45 = 5535$

Для примера $123 \times 45 = 5535$ алгоритм выдаёт правильный результат.

Алгоритм 4: Быстрый столбик

Описание алгоритма

Оптимизированный метод умножения, использующий один аккумулятор t для вычисления суммы вкладов всех пар разрядов в конкретную позицию результата. Перенос обрабатывается один раз для всего разряда, что ускоряет выполнение.

Код реализации

```
def multiply_fast(u, v, b=10):
    """
    Algorithm 4: Fast multiplication
    u, v: strings of digits
    b: base (default 10)
    Returns: product as string
    """
    u_digits = str_to_digits(u)
    v_digits = str_to_digits(v)
    n = len(u_digits)
    m = len(v_digits)

    # Initialize result array
    w_digits = [0] * (m + n)
    t = 0
```

Результат выполнения

```
4. MULTIPLICATION (FAST)
12345 × 6789 = 83810205
Expected: 83810205
Correct: True
```

Рис. 9: Результат быстрого умножения $12345 \times 6789 = 83810205$

Пример $12345 \times 6789 = 83810205$ подтверждает корректность реализации.

Алгоритм 5: Деление

Описание алгоритма

Наиболее сложный алгоритм, реализованный методом длинного деления. Процесс основан на цикле «оценка-исправление»: сначала оценивается цифра частного, затем при необходимости выполняется коррекция.

Код реализации

```
def divide(u, v, b=10):
    """
    Long division algorithm for multi-digit integers
    u, v: strings of digits
    b: base (default 10)
    Returns: (quotient, remainder) as strings
    """
    # Handle division by zero
    if v == "0":
        raise ValueError("Division by zero")

    # If u < v, quotient is 0, remainder is u
    if len(u) < len(v) or (len(u) == len(v) and u < v):
        return "0", u
```

Результат выполнения

```
5. DIVISION
100 ÷ 7 = 14 remainder 2
Expected: 14 remainder 2
Correct: True
```

Рис. 11: Результат деления $100 \div 7 = 14$ *(остаток* 2*)*

Для примера $100 \div 7 = 14$ (остаток 2) алгоритм работает правильно.

Тестирование всех алгоритмов

Код тестирования

Для проверки корректности работы всех алгоритмов была создана функция `test_algorithms`, которая последовательно тестирует каждую операцию.

```
def test_algorithms():
    """Test all algorithms with examples"""

    print("=" * 60)
    print("TESTING MULTIPLE PRECISION ARITHMETIC ALGORITHMS")
    print("=" * 60)

    # Test 1: Addition
    print("\n1. ADDITION")
    u1, v1 = "12345", "56789"
    result1 = add(u1, v1)
    expected1 = str(int(u1) + int(v1))
    print(f"{u1} + {v1} = {result1}")
    print(f"Expected: {expected1}")
    print(f"Correct: {result1 == expected1}")

    # Test 2: Subtraction
    print("\n2. SUBTRACTION")
    u2, v2 = "56789", "12345"
    result2 = subtract(u2, v2)
    expected2 = str(int(u2) - int(v2))
```

Полные результаты тестирования

```
1. ADDITION
12345 + 56789 = 69134
Expected: 69134
Correct: True

2. SUBTRACTION
56789 - 12345 = 44444
Expected: 44444
Correct: True

3. MULTIPLICATION (COLUMN)
123 × 45 = 5535
Expected: 5535
Correct: True
```

Итоги работы

Вывод

В ходе лабораторной работы были успешно реализованы пять алгоритмов целочисленной арифметики многократной точности: сложение, вычитание, умножение столбиком, быстрое умножение и деление. Все алгоритмы протестированы на примерах и работают корректно. Программная реализация позволяет проводить вычисления с числами произвольной длины, что критически важно для криптографических приложений.